

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

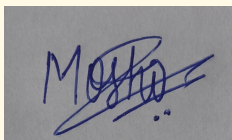
Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Moshika M	Reg. No.:	192424064
Section / Batch:	B.Tech (AI-DS)	Date:	06.08.2026

Code of Conduct

I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student:

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Annexure A – Experiments

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S → NP VP
NP → Det N
VP → V NP
Det → the | a
N → student | teacher | book
V → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

ANSWER:

```
PyCharmMiscProject Version control main
1.py
def parse_cfg(sentence):
    words = sentence.lower().split()
    det = ["the", "a"]
    noun = ["student", "teacher", "book"]
    verb = ["reads", "likes"]
    print("Sentence:", sentence)
    print()
    if len(words) != 5:
        print("Sentence is INVALID.")
        print("\nReason:")
        print("Sentence does not follow the CF6 rule.")
        print("\nOutput:")
        print("Invalid Sentence")
        return
    d1, n1, v, d2, n2 = words
    if d1 in det and n1 in noun and v in verb and d2 in det and n2 in noun:
        print("Sentence is VALID.\n")
        print("Parse Tree\n")
        print("      S")
        print("    /  \")
        print("  NP   VP")
        print(" /  \  /  \")
        print("Det  N V  NP")
        print(" |  | | /  \")
        print(f" {d1} {n1} {v} {d2} {n2}")
        print(f"      |  |")
        print(f"      {d2} {n2}")
        print("\nDictionary Representation:")
        tree = {
            "S": {

```

```

                "NP": {"Det": d1, "N": n1},
                "VP": {
                    "V": v,
                    "NP": {"Det": d2, "N": n2}
                }
            }
        }
        print(tree)
    else:
        print("Sentence is INVALID.")
        print("\nReason:")
        print("Words do not satisfy the grammar rules.")
        print("\nOutput:")
        print("Invalid Sentence")

sentence = input("Enter a sentence: ")
parse_cfg(sentence)
```

```
Run main
/Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/1.py
Enter a sentence: the student reads a book
Sentence: the student reads a book

Sentence is VALID.

Parse Tree

      S
    /  \
  NP   VP
 /  \  /  \
Det  N V  NP
 |  | | /  \
the student reads Det  N
                |  |
                a  book

Dictionary Representation:
{'S': {'NP': {'Det': 'the', 'N': 'student'}, 'VP': {'V': 'reads', 'NP': {'Det': 'a', 'N': 'book'}}}}
```

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he      → singular, third person
she     → singular, third person
it      → singular, third person
they    → plural
runs    → singular verb
writes  → singular verb
run     → plural verb
write   → plural verb
```

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject	Verb	Expected
he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

ANSWER:

```
PyCharmMiscProject Version control main
2.py
1 def check_agreement(subject, verb):
2     subjects = {
3         "he": ("Singular", "Third Person"),
4         "she": ("Singular", "Third Person"),
5         "it": ("Singular", "Third Person"),
6         "they": ("Plural", "")
7     }
8     verbs = {
9         "runs": "Singular Verb",
10        "writes": "Singular Verb",
11        "run": "Plural Verb",
12        "write": "Plural Verb"
13    }
14    print("\nSubject :", subject)
15    print("Verb      :", verb)
16    print()
17    if subject not in subjects or verb not in verbs:
18        print("Agreement Status : FALSE")
19        return
20    number, person = subjects[subject]
21    print("Subject Feature :", number, person)
22    print("Verb Feature      :", verbs[verb])
23    if (number == "Singular" and verbs[verb] == "Singular Verb") or \
24        (number == "Plural" and verbs[verb] == "Plural Verb"):
25        print("\nAgreement Status : TRUE")
26        print("Reason:")
27        print("Subject and verb agree.")
28        return True
29    else:
30        print("\nAgreement Status : FALSE")
Run main
PyCharmMiscProject > 2.py 40:1 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 464 of 1200M
```

```
31    print("Reason:")
32    print("Subject and verb do not agree.")
33    return False
34    sub = input("Enter Subject: ").lower()
35    vb = input("Enter Verb: ").lower()
36    check_agreement(sub, vb)
Run main
PyCharmMiscProject > 2.py 40:1 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 605 of 1400M
```

```
Run main
/Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/2.py
Enter Subject: he
Enter Verb: runs

Subject : he
Verb      : runs

Subject Feature : Singular Third Person
Verb Feature      : Singular Verb

Agreement Status : TRUE
Reason:
Subject and verb agree.

Process finished with exit code 0
PyCharmMiscProject > 2.py 40:1 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 571 of 1400M
```

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

```
S → NP VP      [1.0]
NP → John       [0.6]
NP → Mary       [0.4]
VP → runs       [0.5]
VP → walks      [0.5]
```

Probability Formula

```
P(Sentence)
=
P(S → NP VP)
× P(NP)
× P(VP)
```

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if the sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence	Expected Probability
John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

ANSWER:

```
PyCharmMiscProject Version control main 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 def sentence_probability(sentence): 1 usage np = { "John": 0.6, "Mary": 0.4 } vp = { "runs": 0.5, "walks": 0.5 } words = sentence.split() print("\nSentence:", sentence) if len(words) != 2: print("Invalid Sentence") return subject, verb = words if subject not in np or verb not in vp: print("\nError:") print(f'"{subject}" is not present in the grammar.') print("\nFinal Probability = 0.00") return probability = 1.0 * np[subject] * vp[verb] print("\nParse:") print("S → NP VP\n") print(f'NP → {subject} [{np[subject]}]') print(f'VP → {verb} [{vp[verb]}]') print("\nProbability Calculation") print("\nP(Sentence)") print("= P(S→NP VP) × P(NP) × P(VP)") print(f"= 1.0 × {np[subject]} × {vp[verb]}") print(f"= {probability:.2f}") Run main 33:31 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 508 of 1200M
```

```
31 print(f'\nFinal Probability = {probability:.2f}') 32 sentence = input("Enter sentence: ") 33 sentence_probability(sentence) Run main 23:25 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 502 of 1400M
```

```
Run main /Users/moshika/PyCharmMiscProject/.venv/bin/python /Users/moshika/PyCharmMiscProject/3.py Enter sentence: John runs Sentence: John runs Parse: S → NP VP NP → John [0.6] VP → runs [0.5] Probability Calculation P(Sentence) = P(S→NP VP) × P(NP) × P(VP) = 1.0 × 0.6 × 0.5 = 0.30 Final Probability = 0.30 Process finished with exit code 0 PyCharmMiscProject 3.py 23:25 LF UTF-8 4 spaces Python 3.9 (PyCharmMiscProject) 465 of 1400M
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____